1. Цель и результат работы

Цель работы — создать веб-приложение более чем с одной страницей, настроить VPN, интегрировать авторизацию по OAuth 2.0 и обеспечить доступ к приложению с клиентской машины только при активном VPN. В результате созданы четыре HTML-страницы, вход через Google OAuth 2.0 и проверен отказ доступа при отключённом WireGuard.

2. Среда и схема работы

Использовались две виртуальные машины Lubuntu в VirtualBox: серверная и клиентская. VPN-подключение выполнено по WireGuard. Веб-сервер и OAuth2 Proxy слушают только loopback-интерфейс серверной ВМ, а SSH доступен на VPN-адресе сервера.

Узел / компонент	Адрес или порт	Назначение
Сервер WireGuard	10.8.0.1/24	VPN-шлюз и хост приложения
Клиент WireGuard	10.8.0.2/24	Клиентская ВМ; проверка ping и доступ к сайту
WireGuard	UDP 51820	Зашифрованный VPN-туннель между ВМ
Python http.server	127.0.0.1:8080	Выдача четырёх статических HTML-страниц
oauth2-proxy	127.0.0.1:4180	OAuth-вход и защита веб-приложения
SSH	10.8.0.1:22	Локальный порт-форвардинг, доступный через WireGuard

Клиент открывает в браузере http://localhost:4180. Локальный SSH-форвард передаёт соединение на 127.0.0.1:4180 серверной ВМ по WireGuard. OAuth2 Proxy перенаправляет неавторизованного пользователя на Google, а после входа передаёт запрос локальному веб-серверу на порту 8080.

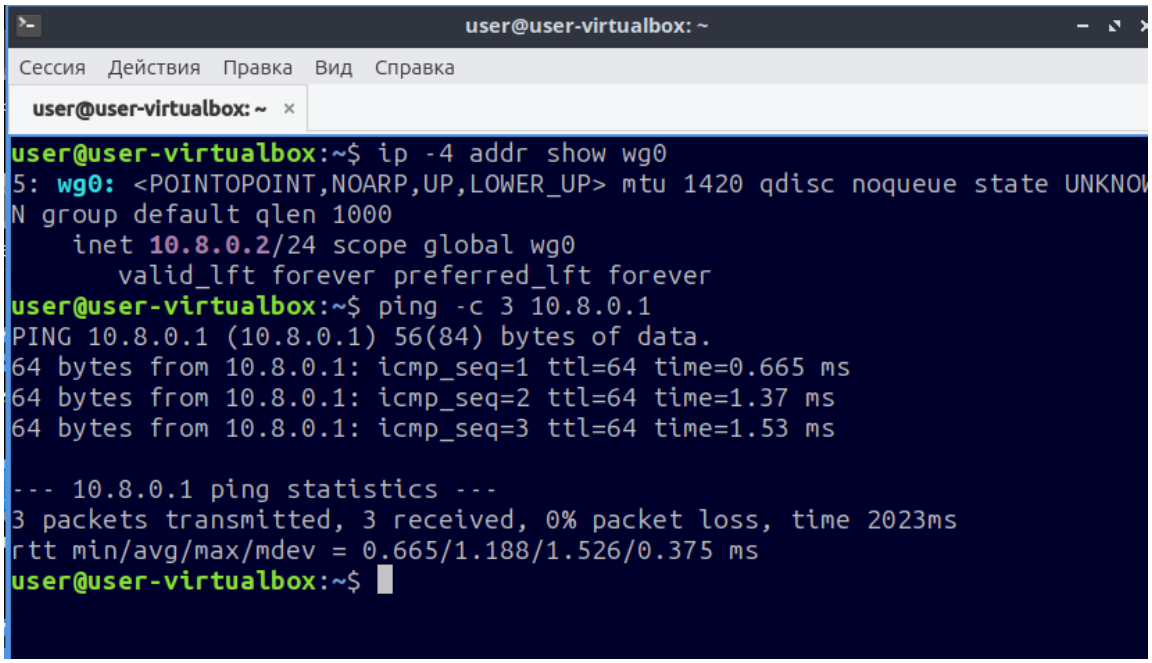


Рисунок 1 — адрес клиента WireGuard 10.8.0.2 и успешный ping до сервера 10.8.0.1

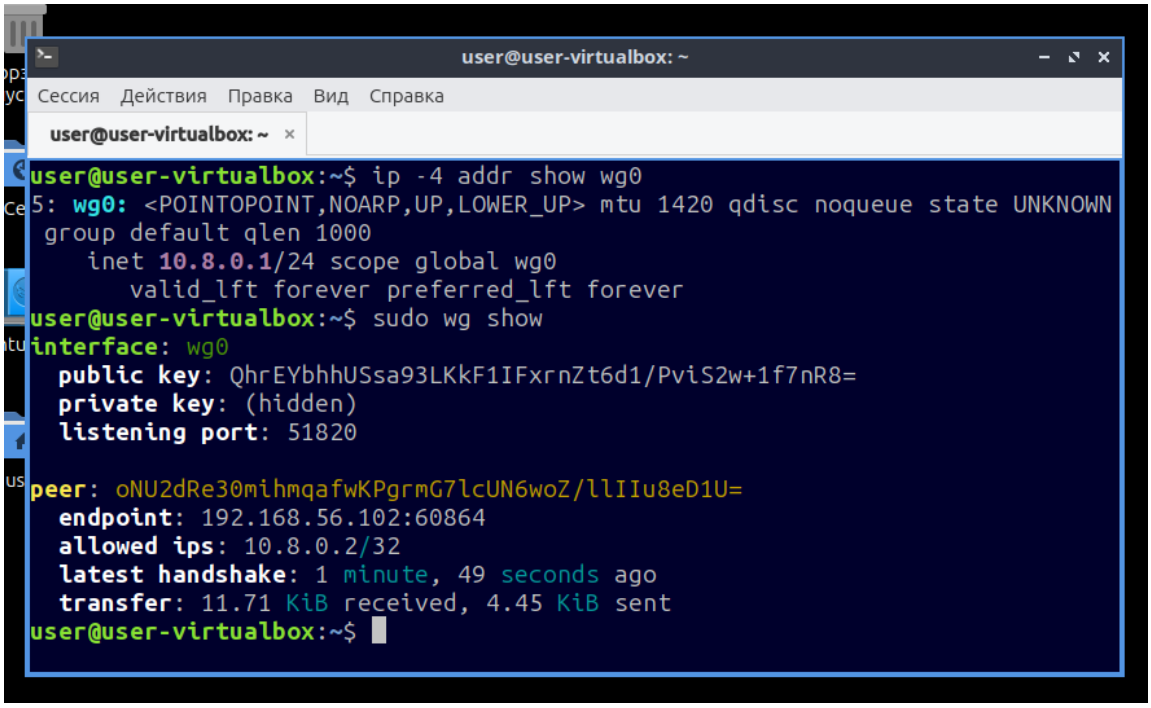


Рисунок 2 — интерфейс wg0, peer, handshake и обмен данными на сервере

3. Создание веб-приложения

Создан каталог ~/vpn-web/site с четырьмя HTML-файлами. Страницы содержат различающиеся заголовки и ссылки навигации, поэтому пользователь может переходить между ними. Для выдачи статических файлов использован встроенный сервер Python без дополнительного веб-фреймворка. Он запущен с привязкой к 127.0.0.1:8080, поэтому напрямую из сети клиента обратиться к нему нельзя.

Файл	Содержимое
index.html	Главная страница и ссылки на остальные разделы
page2.html	Вторая страница приложения
page3.html	Третья страница приложения
page4.html	Четвёртая страница приложения

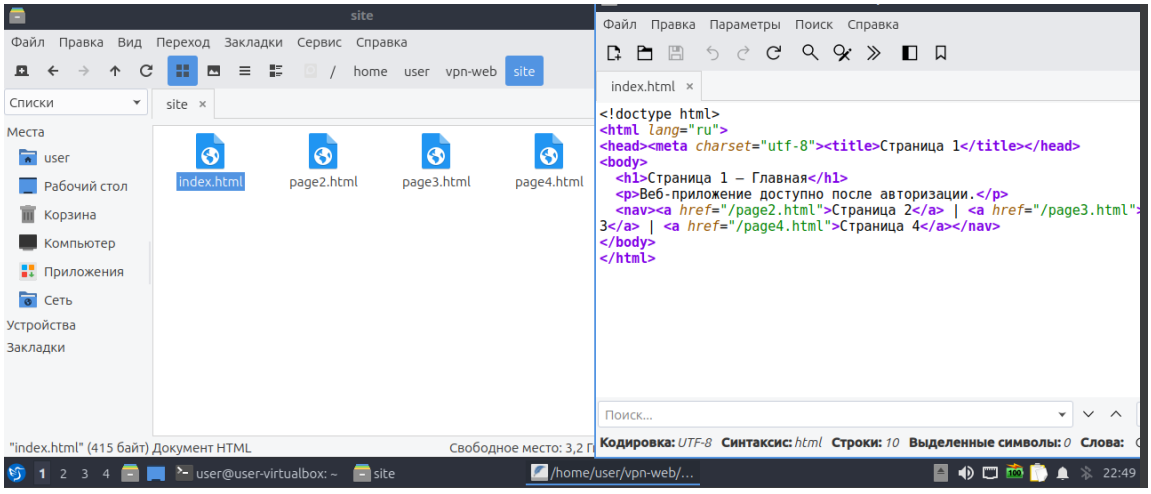


Рисунок 3 — каталог с четырьмя HTML-файлами и исходный код главной страницы

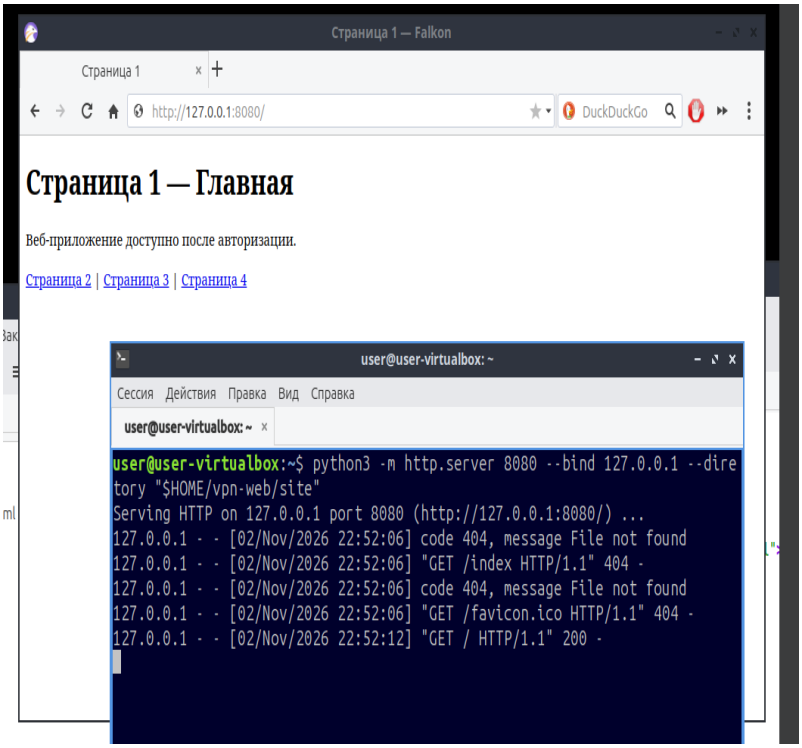


Рисунок 4 — работа локального Python HTTP-сервера и отображение главной страницы

4. Настройка OAuth 2.0

В Google Cloud Console создан проект и настроено приложение OAuth. Для клиентского приложения типа Web application задан URI возврата http://localhost:4180/oauth2/callback. В режиме тестирования указан тестовый пользователь. OAuth2 Proxy версии 7.8.1 использует Google как провайдера, принимает callback на localhost:4180 и передаёт только авторизованные запросы на локальный upstream 127.0.0.1:8080.

При запросе без сессии пользователь попадает на страницу входа OAuth2 Proxy, затем проходит аутентификацию в Google. После возврата на callback Proxy создаёт cookie сессии и разрешает доступ к страницам сайта.

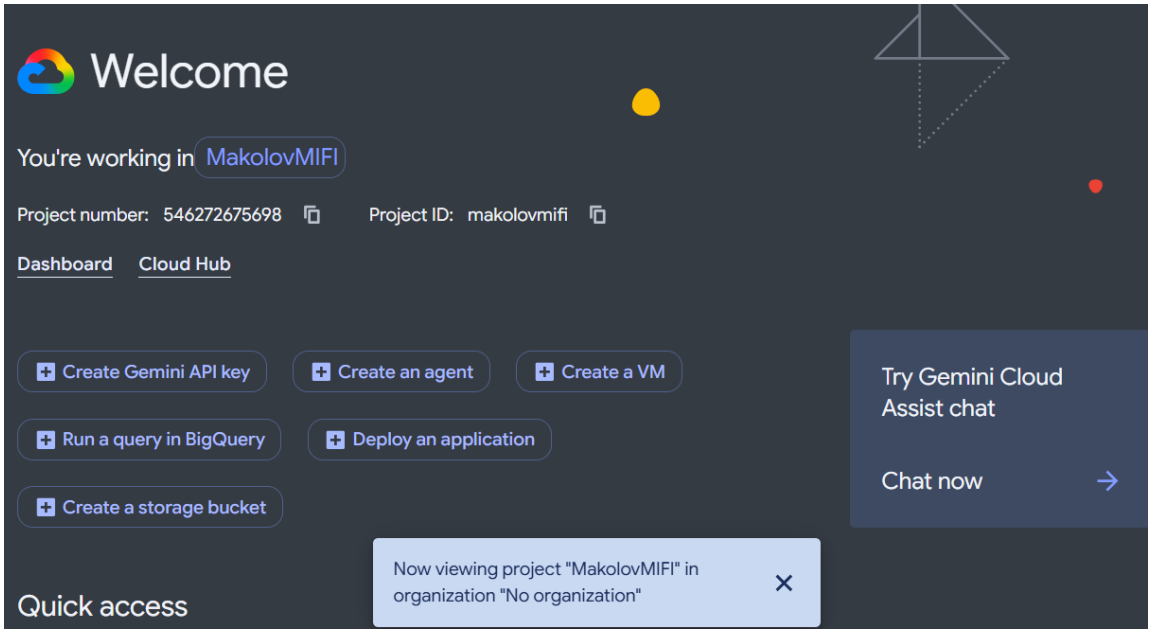


Рисунок 5 — созданный проект Google Cloud для настройки OAuth

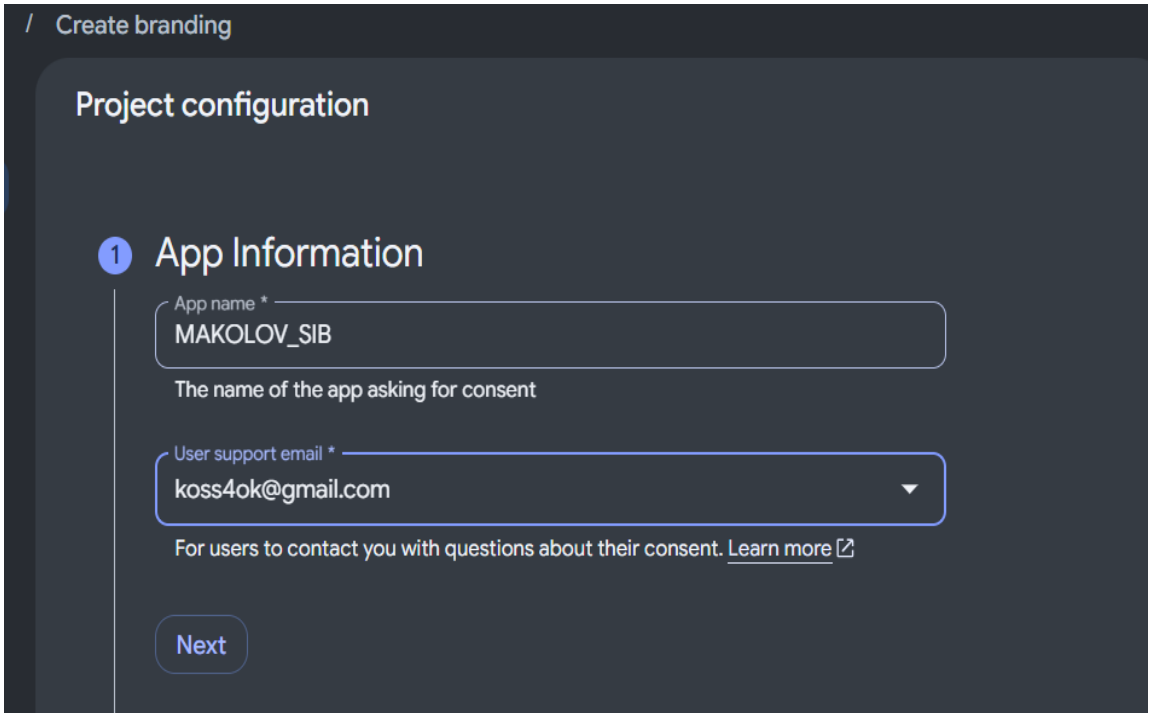


Рисунок 6 — настройка названия приложения и контактной информации OAuth

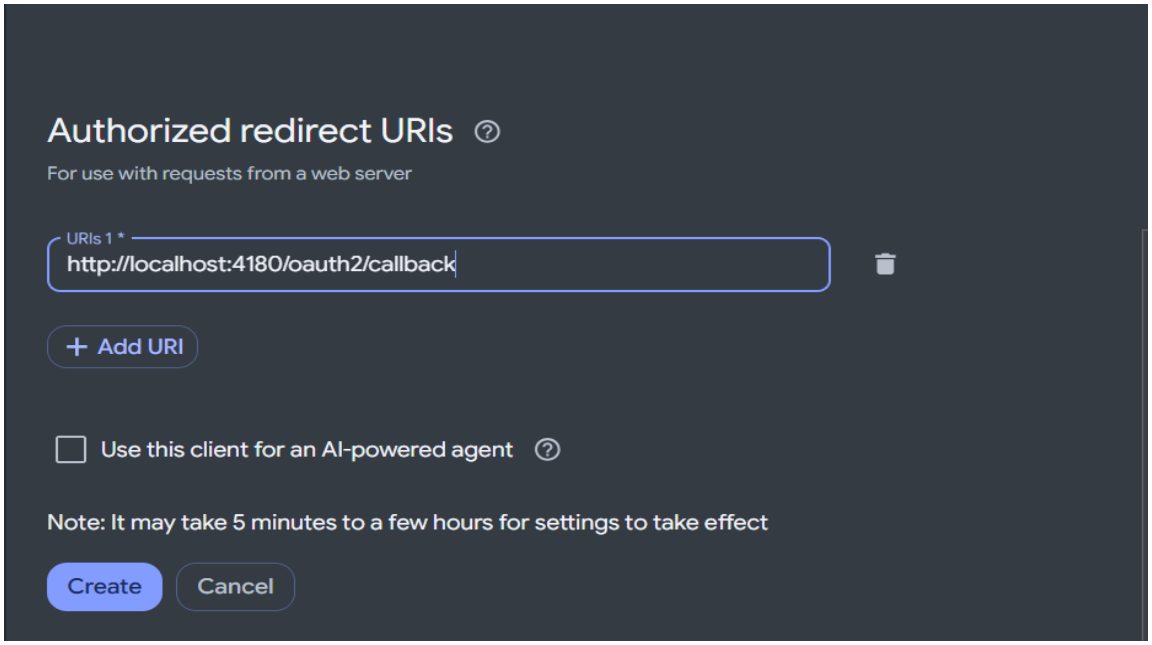


Рисунок 7 — URI возврата OAuth-клиента

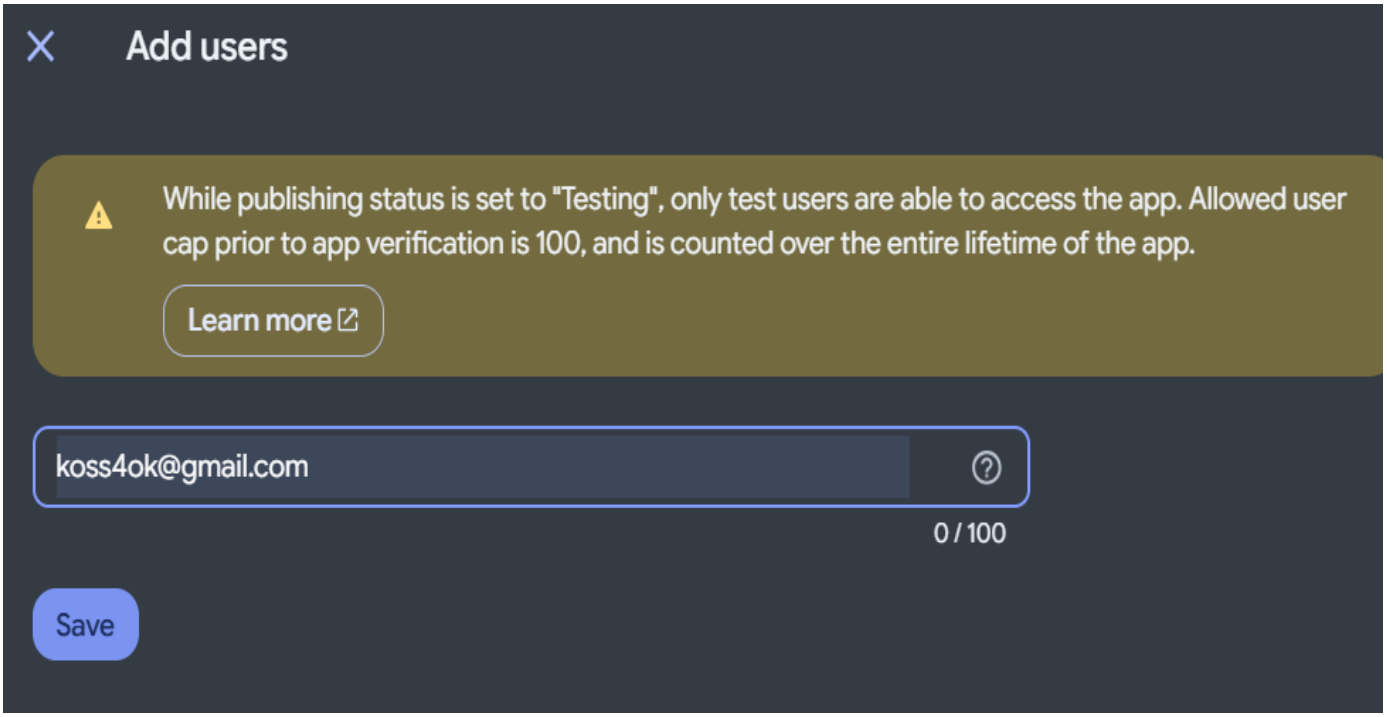


Рисунок 8 — добавление тестового пользователя OAuth-приложения

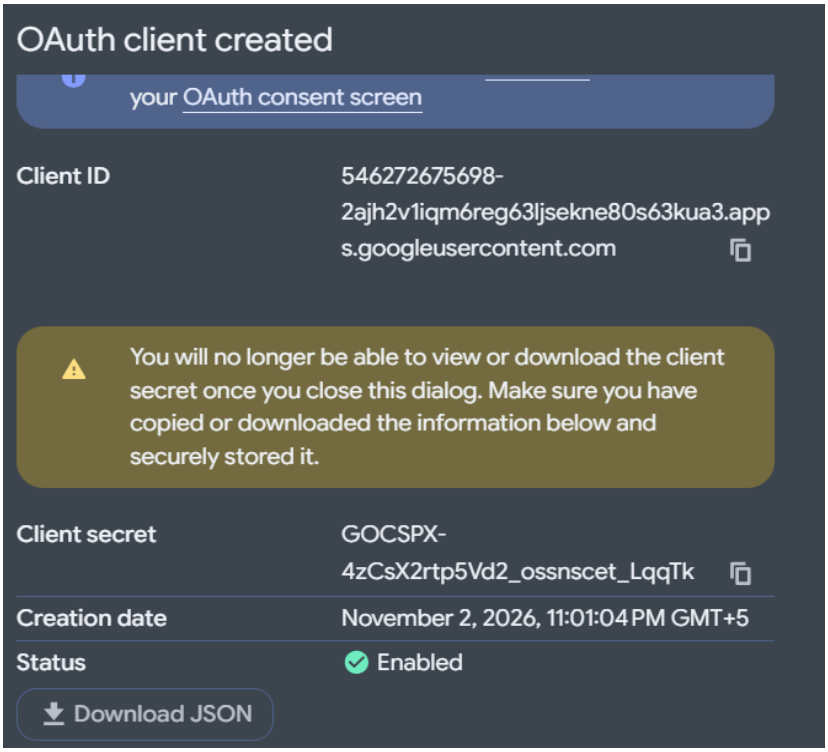


Рисунок 9 — создание OAuth-клиента типа Web application

5. Установка и запуск OAuth2 Proxy

OAuth2 Proxy установлен на серверной ВМ как отдельный исполняемый файл. В конфигурации указаны провайдер Google, callback URI, локальный upstream, разрешённый домен электронной почты и адрес прослушивания 127.0.0.1:4180. Секреты используются службой для OAuth-обмена и cookie-сессии; значения не приводятся в тексте отчёта.

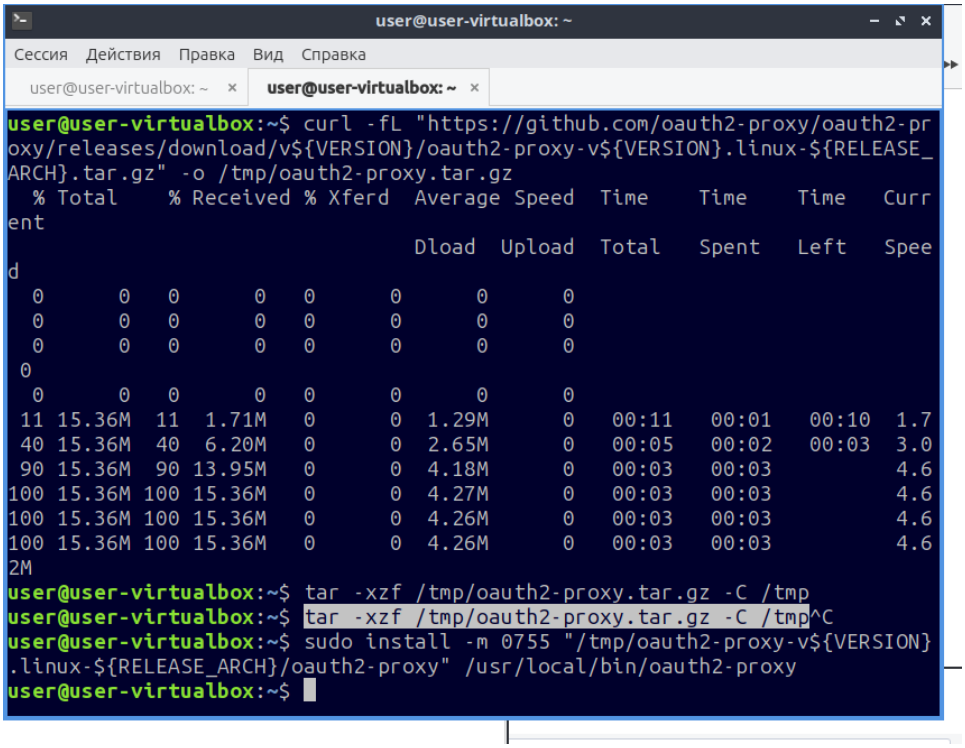


Рисунок 10 — загрузка дистрибутива OAuth2 Proxy

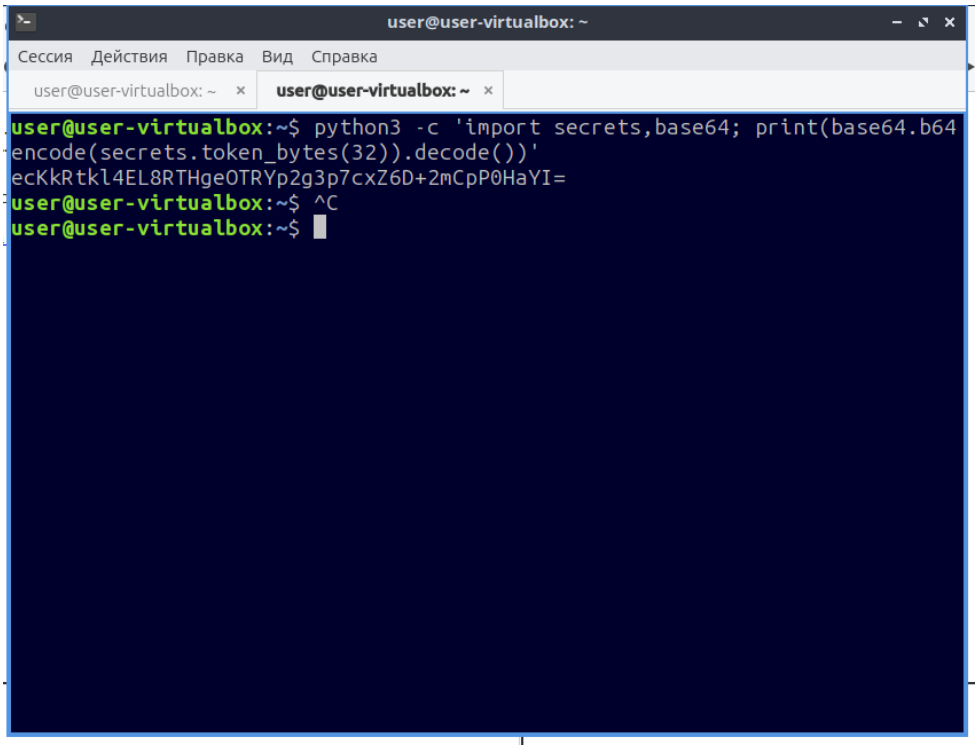


Рисунок 11 — генерация случайного секрета cookie

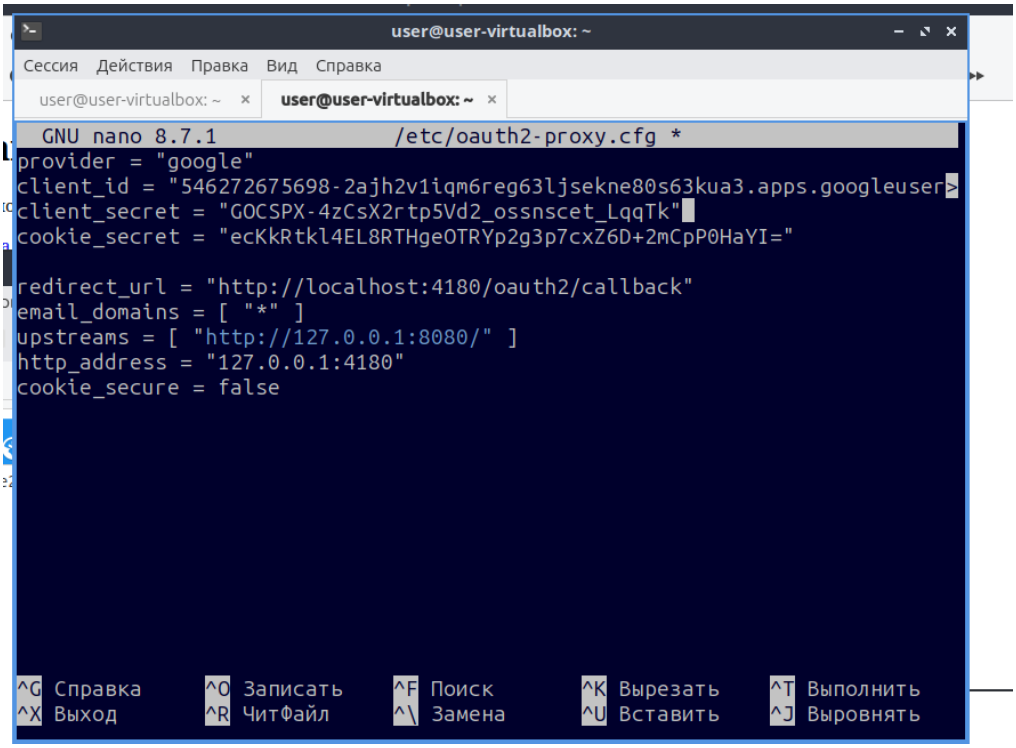


Рисунок 12 — конфигурация OAuth2 Proxy с upstream и параметрами Google OAuth

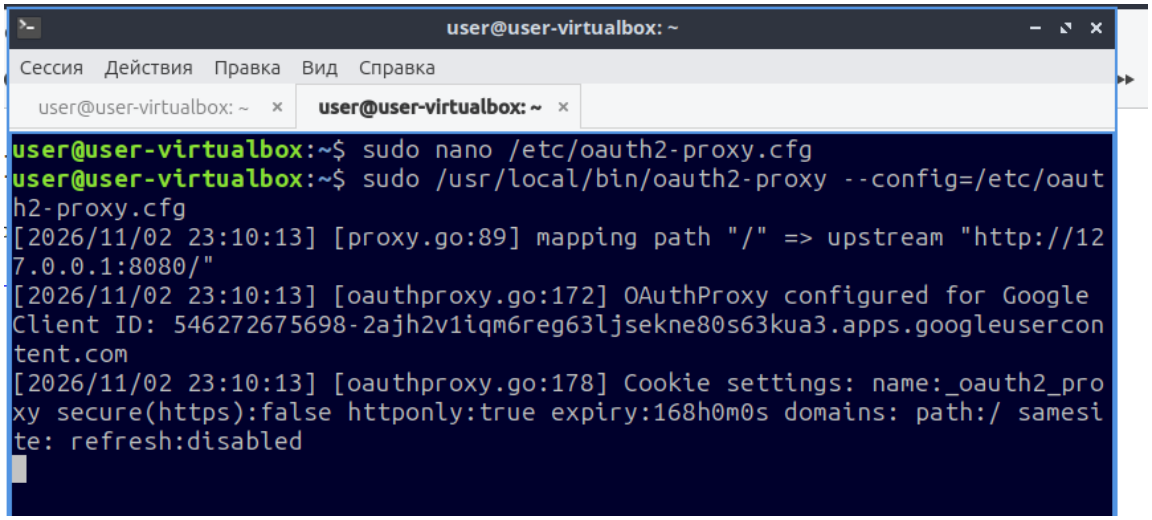


Рисунок 13 — запуск OAuth2 Proxy и отображение настроенного upstream

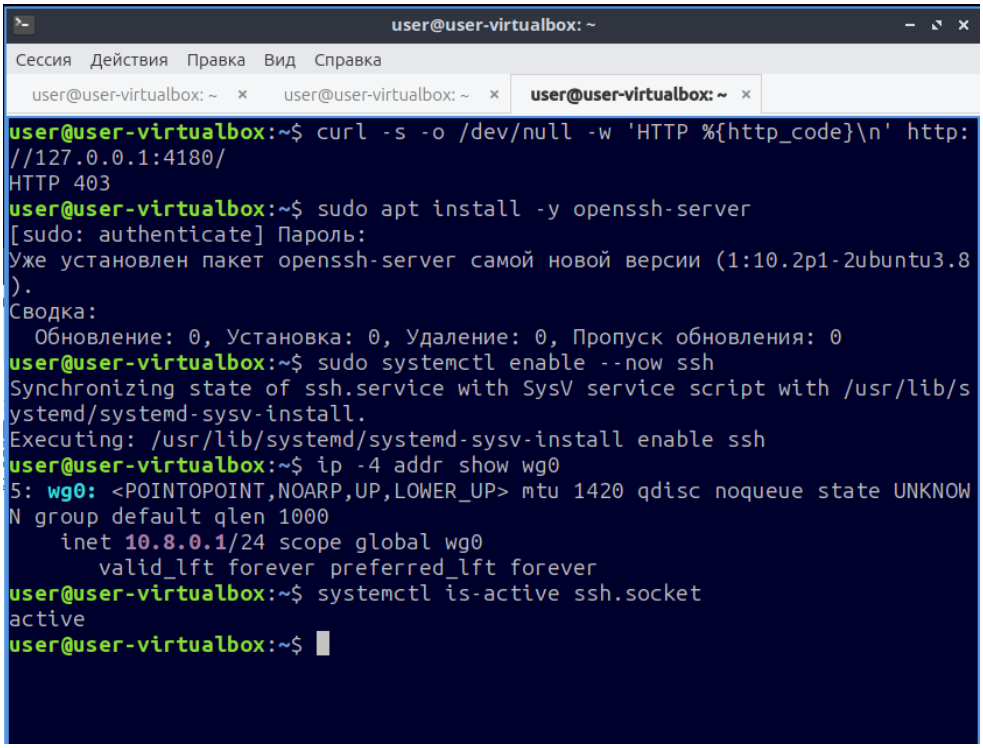


Рисунок 14 — проверка неавторизованного запроса и запуск SSH-службы

6. Ограничение доступа через WireGuard

Чтобы доступ к приложению проходил только по VPN, SSH-сокет сервера настроен на прослушивание адреса 10.8.0.1:22. Веб-сервер и OAuth2 Proxy доступны только на loopback серверной ВМ. На клиенте используется команда ssh -N -L 4180:127.0.0.1:4180 user@10.8.0.1. Таким образом, запрос к веб-приложению передаётся через SSH-туннель, который устанавливается к WireGuard-адресу сервера.

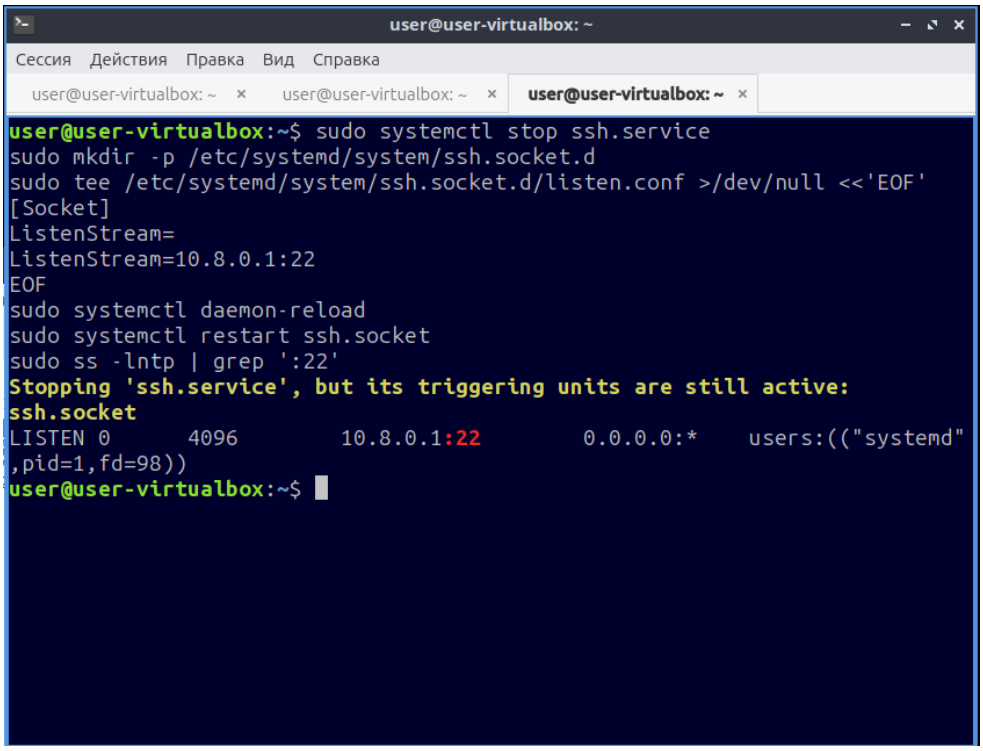


Рисунок 15 — SSH-сокет слушает только VPN-адрес сервера 10.8.0.1:22

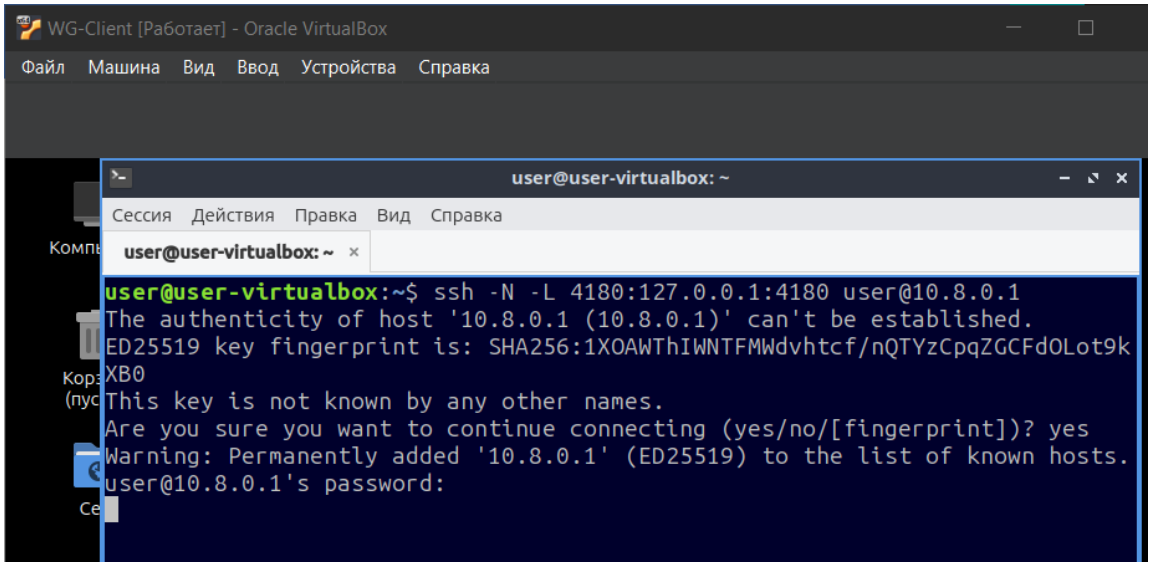


Рисунок 16 — установка SSH-туннеля с клиентской ВМ к серверу по WireGuard

7. Проверка авторизации и страниц приложения

При открытии локального адреса клиента отображается страница OAuth2 Proxy. После выбора Google и входа в тестовую учётную запись браузер возвращается на главную страницу. Навигация позволяет открывать страницы 2–4; в исходном каталоге присутствуют все четыре HTML-файла. Скриншоты демонстрируют вход и страницы приложения.

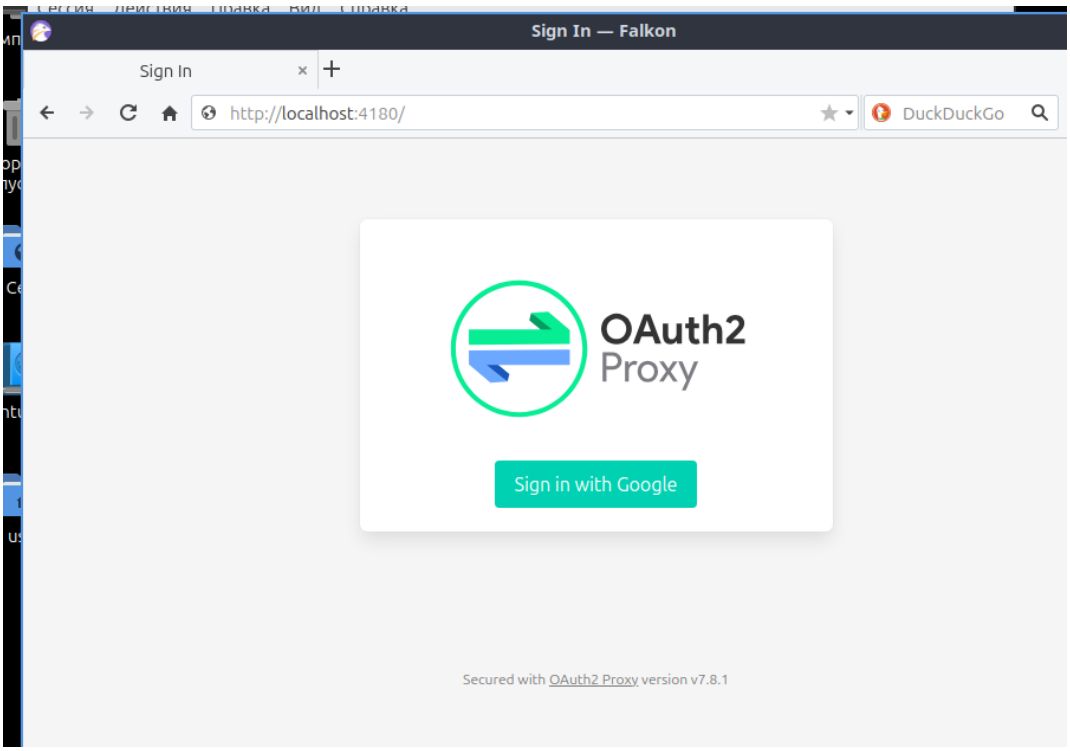


Рисунок 17 — страница входа OAuth2 Proxy с кнопкой Google

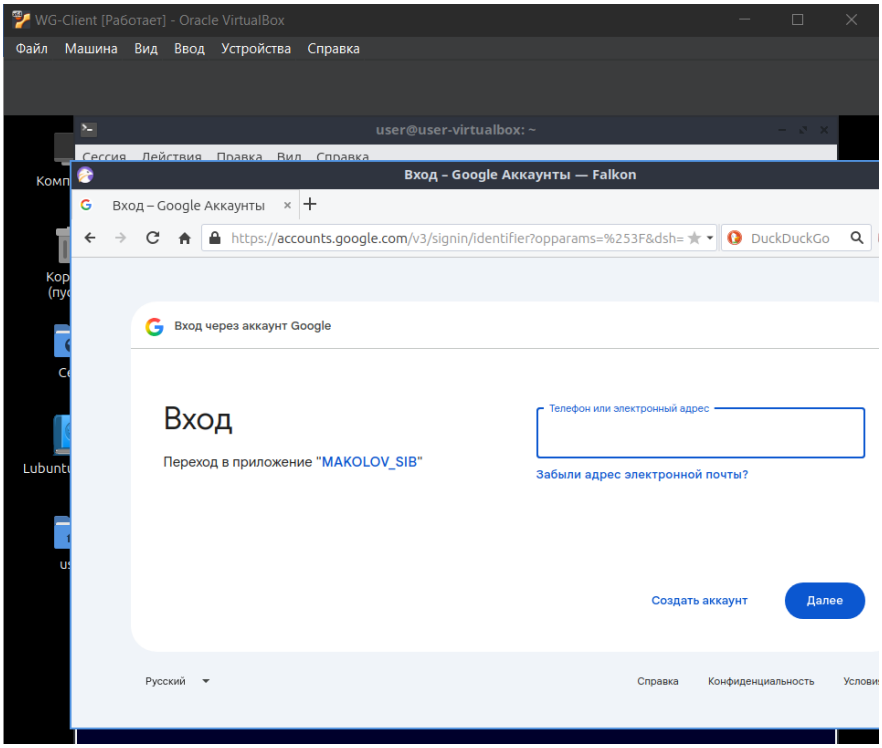


Рисунок 18 — страница авторизации Google для приложения

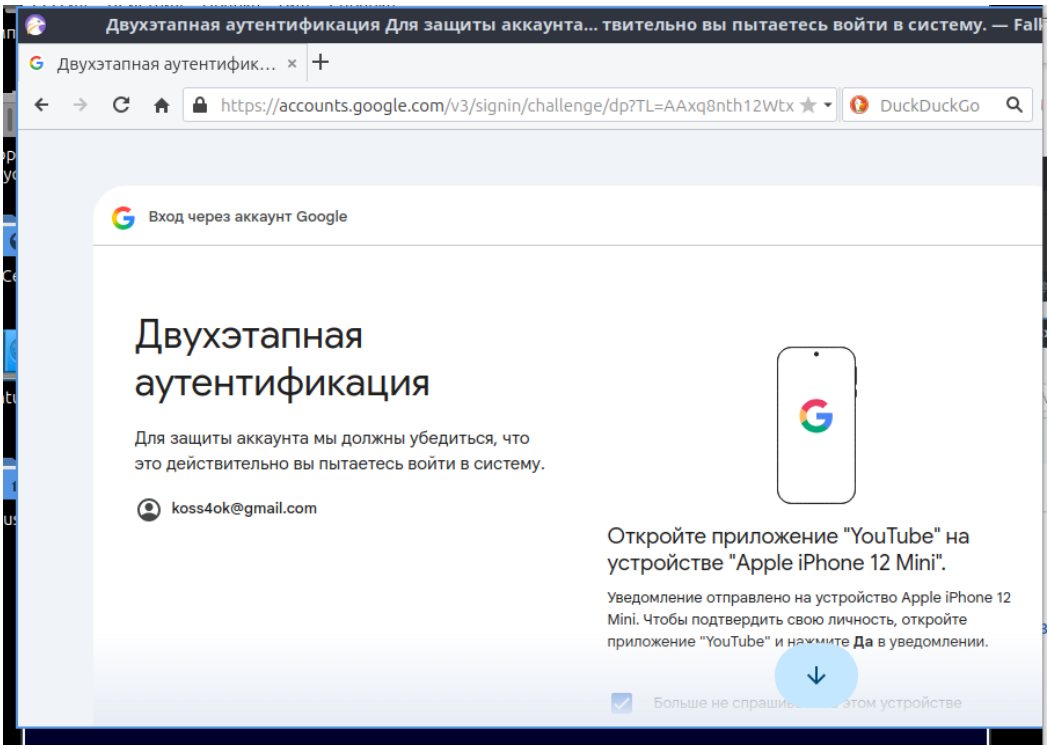


Рисунок 19 — этап дополнительного подтверждения учётной записи Google

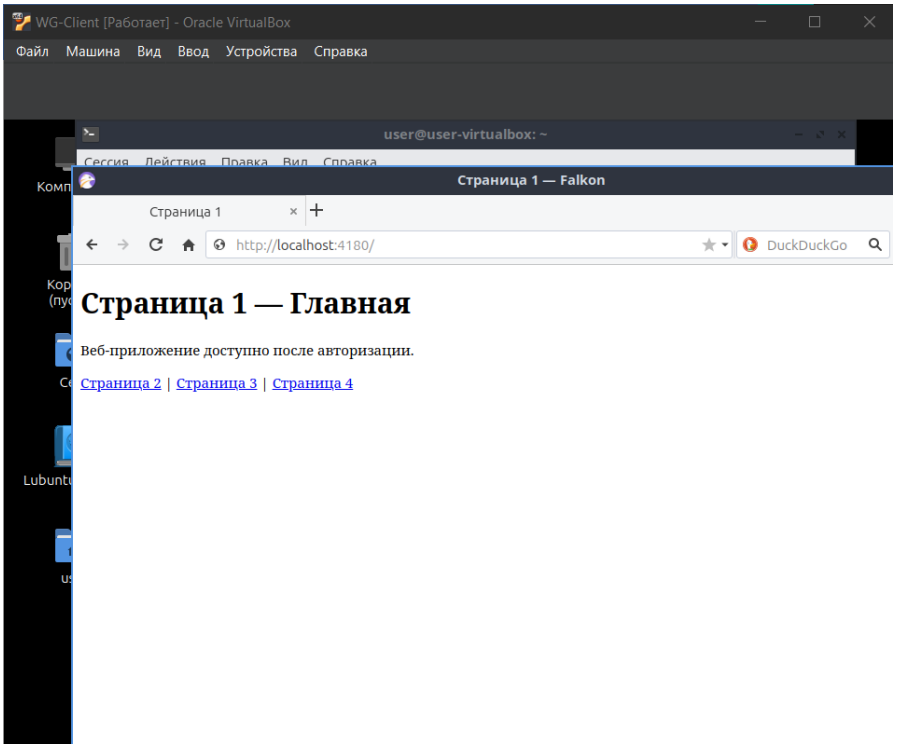


Рисунок 20 — главная страница после успешной авторизации

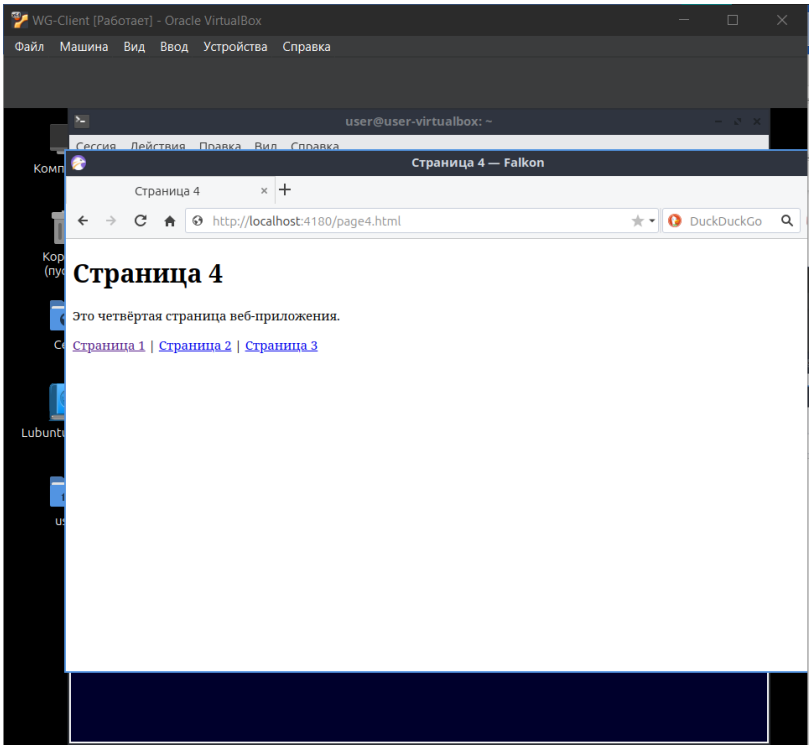


Рисунок 21 — четвёртая страница после успешной авторизации

8. Проверка недоступности без VPN

На клиенте WireGuard остановлен командой wg-quick down wg0. После отключения VPN подключение SSH к адресу 10.8.0.1:22 не устанавливается, а браузер не может открыть localhost:4180, поскольку SSH-форвард отсутствует. После повторного включения WireGuard и запуска туннеля доступ восстанавливается. Проверка подтверждает, что приложение доступно с клиентской машины только при активном VPN.

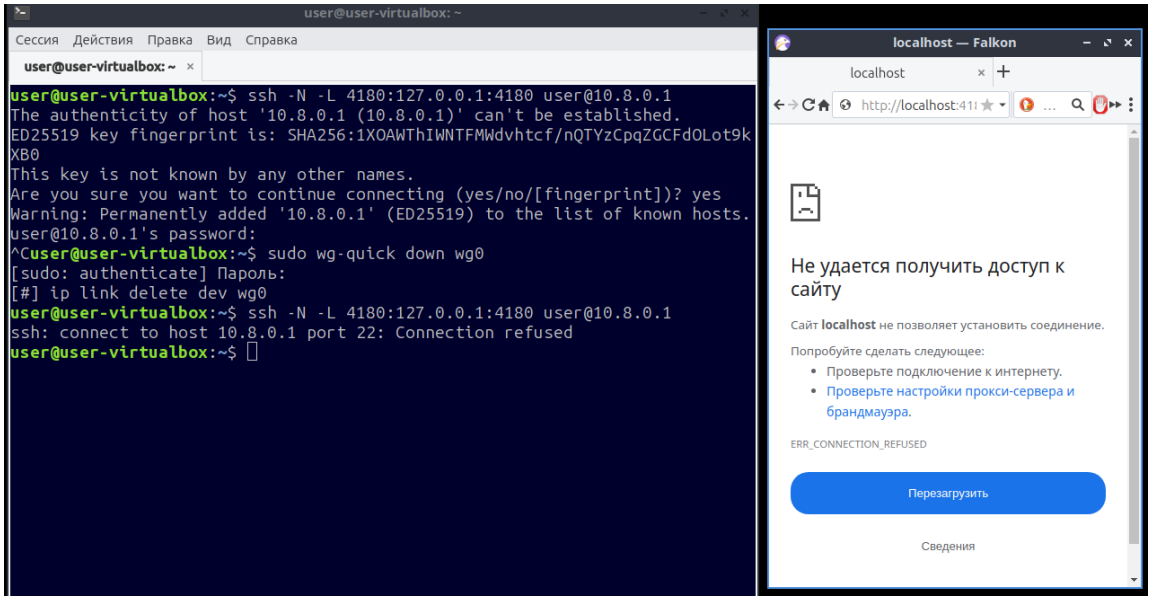


Рисунок 22 — неудачная попытка SSH-подключения и отказ браузера при выключенном WireGuard

9. Вывод

Создано многостраничное статическое веб-приложение из четырёх HTML-страниц. Доступ защищён OAuth 2.0 через OAuth2 Proxy и Google. WireGuard обеспечивает VPN-связность клиентской и серверной ВМ; SSH-перенаправление и привязка служб к loopback/VPN-адресам исключают доступ к веб-приложению при отключённом VPN. Проверены успешный вход, переход между страницами и отказ доступа без VPN.